Tutorial - 2

Comparison: Active vs Passive Intruder

Feature	Active Intruder	Passive Intruder
Definition	An intruder who **modifies, injects, or deletes messages** in a communication system.	An intruder who **only listens to the communication** without altering messages.
Goal	Disrupt or alter data to **cause harm** or **steal information**.	Gather confidential information **without detection**.
Actions	- Message modification	

- Replay attacks
- Man-in-the-middle (MITM) attacks
- Impersonation | - Eavesdropping
- Traffic analysis
- Intercepting messages |

| **Detection** | **Easier to detect** since it modifies data and may cause failures. | **Harder to detect** because no visible changes occur. |

| **Example** | A hacker intercepts a login request and **changes the password** before sending it to the server. | A hacker **sniffs network traffic** to steal login credentials. |

How to Identify Cipher Type (Substitution vs Transposition) at a Glance

Substitution Cipher

- Each letter is **replaced** with another letter, symbol, or number.
- Letter **positions remain the same**.

  Example:

  - Plaintext: HELLO
  - Substituted (Caesar Cipher with shift 3): KHOOR

Transposition Cipher

- The **positions of letters are shuffled** without replacing them.
- Frequency of letters **remains the same** as plaintext.

  Example:

  - Plaintext: HELLO
  - Transposed: LOHLE

👉 Quick Identification:

- If letter frequencies **match the original plaintext**, it's likely **Transposition**.
- If letter frequencies are **completely changed**, it's **Substitution**.

Example of Transposition Cipher using "JAHANGIRU" as the Key

Step 1: Arrange Plaintext in a Grid (Columnar Transposition)

Take a **plaintext message**:

```
ATTACKATDAWN
```

Arrange it in **columns** under the key "JAHANGIRU" (9 letters):

J	A	H	A	N	G	I	R
A	T	T	A	C	K	A	T
A	W	N	X	X	X	X	X

(X is added to fill empty spaces.)

Step 2: Rearrange Columns by Key Order

Alphabetical order of "JAHANGIRU" gives column sequence:

```
A (2), A (4), G (6), H (3), I (7), J (1), N (5), R (8), U (9)
```

Read **column-wise** in this order:

```
TAXTA
AXXXX
TKNAD
```

Step 3: Write the Ciphered Text

```
TAXTAAXXXXTKNAD
```

👉 This is the **transposition ciphered text** for "ATTACKATDAWN" using "JAHANGIRU" as the key! 🚀

Give basic principle of asymmetric key cryptography.

Basic Principle of Asymmetric Key Cryptography

Asymmetric key cryptography, also known as **public-key cryptography**, uses two mathematically related keys:

1. **Public Key**: Used for encryption. This key is publicly shared.
2. **Private Key**: Used for decryption. This key is kept secret.

Working Principle

1. The **sender encrypts** the message using the recipient's **public key**.
2. The **recipient decrypts** the message using their **private key**.
3. It ensures **confidentiality** (only the intended recipient can decrypt) and **authentication** (via digital signatures).

Taking p=3 and q=11 determines all the parameters of RSA then verify encryption and decryption of the vector V=[7,12,3].

RSA Key Generation with $p = 3$ and $q = 11$

Step 1: Compute n

$$n = p \times q = 3 \times 11 = 33$$

Step 2: Compute Euler's Totient Function $\phi(n)$

$$\phi(n) = (p-1) \times (q-1) = (3-1) \times (11-1) = 2 \times 10 = 20$$

Step 3: Choose Public Key Exponent e

- e must be **coprime** to $\phi(n) = 20$.
- Let's choose $e = 3$ (since $\gcd(3, 20) = 1$).

Step 4: Compute Private Key d

- d is the modular inverse of e modulo $\phi(n)$, meaning:

$$d \times e \equiv 1 \bmod \phi(n)$$

$$d \times 3 \equiv 1 \bmod 20$$

Using the **Extended Euclidean Algorithm**, we find:

$d = 7$

Public and Private Key Pair

- **Public Key**: $(e, n) = (3, 33)$
- **Private Key**: $(d, n) = (7, 33)$

RSA Encryption & Decryption for $V = [7, 12, 3]$

Encryption: $C = V^e \bmod n$

1. $C_1 = 7^3 \bmod 33$

$7^3 = 343$

$343 \bmod 33 = 343 - (33 \times 10) = 343 - 330 = 13$

2. $C_2 = 12^3 \bmod 33$

$12^3 = 1728$

$1728 \bmod 33 = 1728 - (33 \times 52) = 1728 - 1716 = 12$

3. $C_3 = 3^3 \bmod 33$

$3^3 = 27$

$27 \bmod 33 = 27$

So, the **encrypted vector** is:

$C = [13, 12, 27]$

Decryption: $M = C^d \bmod n$

1. $M_1 = 13^7 \bmod 33$

$13^7 \bmod 33 = 13^{(3+3+1)} \bmod 33$

Breaking it down:

$13^3 \bmod 33 = 2197 \bmod 33 = 13$

$(13^3)^2 \bmod 33 = 13^6 \bmod 33 = 169 \bmod 33 = 4$

$13^7 = 13^6 \times 13 \bmod 33 = 4 \times 13 = 52$

$52 \bmod 33 = 7$

2. $M_2 = 12^7 \bmod 33$

$12^3 = 1728 \bmod 33 = 12$

$(12^3)^2 = 12^6 \bmod 33 = 144 \bmod 33 = 12$

$12^7 = 12^6 \times 12 \bmod 33 = 12 \times 12 = 144$

$144 \bmod 33 = 12$

3. $M_3 = 27^7 \bmod 33$

$27^3 = 19683 \bmod 33 = 27$

$27^6 \bmod 33 = 27$

$27^7 = 27^6 \times 27 \bmod 33 = 27 \times 27 = 729$

$729 \bmod 33 = 3$

So, the **decrypted vector** is:

$M = [7, 12, 3]$

Since **M matches the original plaintext**, the encryption and decryption process is verified! ✅ 🎉

Comparison Between Conventional and Digital Signatures

Aspect	Conventional Signature	Digital Signature
Inclusion	Part of the document (physically signed)	A separate cryptographic entity attached to the document
Relationship	Signature is directly linked to the signer's handwriting	Signature is mathematically linked to the signer's private key
Duplicity	Can be easily forged by copying	Nearly impossible to duplicate without the private key

Arrangement of Digital Signature using SHA-1 and RSA

Steps for Digital Signature Generation

1. **Hash the message** using SHA-1.
2. **Encrypt the hash** using the sender's private RSA key.
3. **Attach the encrypted hash** (digital signature) to the document.

Steps for Digital Signature Verification

1. **Decrypt the signature** using the sender's public RSA key to get the hash.
2. **Hash the received message** again using SHA-1.
3. **Compare both hashes**; if they match, the signature is valid.

Arrangement Diagram of Digital Signature with SHA-1 & RSA

Digital Signature Generation

```
Message → SHA-1 → Hash → RSA Private Key → Digital Signature
```

Digital Signature Verification

```
Received Message → SHA-1 → Hash'
Digital Signature → RSA Public Key → Hash
Compare: Hash == Hash' → Valid/Invalid
```

Python Implementation of Digital Signature using SHA-1 & RSA

```
from Crypto.PublicKey import RSA
from Crypto.Hash import SHA1
from Crypto.Signature import pkcs1_15

# Generate RSA key pair
key = RSA.generate(1024)
private_key = key
public_key = key.publickey()

# Original message
message = b"Hello, this is a digitally signed message."

# Step 1: Generate Hash using SHA-1
hash_obj = SHA1.new(message)

# Step 2: Sign the hash using the private key
signature = pkcs1_15.new(private_key).sign(hash_obj)

# Verification
try:
    pkcs1_15.new(public_key).verify(hash_obj, signature)
    print("Signature is valid!")
except ValueError:
    print("Signature is invalid!")
```

This implementation:

- Uses **SHA-1** to generate the hash.
- Uses **RSA private key** to sign the hash.
- Uses **RSA public key** to verify the signature.

🔒 **This ensures message authenticity and integrity!** 🚀